

Lộ trình (Roadmap)

PHỤ THUỘC Không có — chương này nên đọc trước Phần A, rồi quay lại sau mỗi giai đoạn. Nó là bản đồ chứ không phải nội dung.

TÓM TẮT

Chương này nói thứ tự nên đi, và quan trọng hơn, *lý do* cho từng thứ tự. Năm giai đoạn: nền tảng không rút ngắn được; đọc hệ thống thật; hợp nhất với hiệu chuẩn; đối mặt thực tế; và cuối cùng là nghiên cứu. Điểm khác biệt của chương so với một danh sách đọc thông thường nằm ở hai chỗ. Thứ nhất, mỗi thứ tự đều có lý do suy ra từ *cấu trúc phụ thuộc* của lĩnh vực — Lie group phải trước bundle adjustment vì BA cần Jacobian trên đa tạp; observability phải sau BA vì nó là câu hỏi về ma trận của BA. Những quan hệ ấy không đổi theo thời gian, nên chương này gần như không hết hạn. Thứ hai, và là luận điểm riêng của chương: *giai đoạn bốn — tự dựng dữ liệu và đo thất bại — bị bỏ qua nhiều nhất và có giá trị cao nhất*. Phần lớn người học nhảy từ giai đoạn ba sang giai đoạn năm, và hậu quả là họ cải tiến những thứ không phải vấn đề. Nếu chỉ nhớ một điều: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ*.

1.1 Vì sao thứ tự quan trọng

Một lộ trình có thể chỉ là một danh sách sắp xếp tùy tiện. Lộ trình này thì không, và lý do đáng nói vì nó cũng là lý do chương này không hết hạn.

Lĩnh vực này có một *cấu trúc phụ thuộc* thật: một số thứ không hiểu được nếu chưa hiểu thứ khác, và quan hệ ấy là quan hệ logic chứ không phải quy ước sự phạm.

Ba ví dụ cụ thể từ chính cây này. *Lie group phải đến trước bundle adjustment*, vì BA cần Jacobian của phần dư theo tư thế, và tư thế sống trên đa tạp — không có A/03 thì công thức ở A/04 là hộp đen. *Observability phải đến sau BA*, vì toàn bộ A/05 là một câu hỏi về ma trận thông tin mà A/04 dựng lên. *Hiệu chuẩn phải đến sau cả hai*, vì C/14 chỉ có nghĩa khi đã có khung MAP để thả biến vào và đã có lý thuyết kích thích để biết khi nào thả được.

Những quan hệ ấy không phụ thuộc vào công nghệ hiện tại. Chúng vẫn đúng nếu mọi framework trong E/23 biến mất.

Có một nguyên tắc thứ hai chi phối lộ trình, ít hiển nhiên hơn: *học một thứ trước khi cần nó thì quên; học nó đúng lúc cần thì nhớ*. Đây là lý do lộ trình xen kẽ lý thuyết với thực hành thay vì học hết lý thuyết rồi mới làm — và là lý do mỗi chương trong cây có một mini project.

1.2 Giai đoạn 1 — Nền tảng, không rút ngắn được

Nội dung: A/01 (hình học đa thị) → A/02 (mô hình camera) → A/03 (Lie group) → A/04 (MAP và BA) → A/05 (observability) → A/06 (bất định).

Vì sao thứ tự này: đã giải thích ở mục 1 cho ba quan hệ chính. Thêm hai quan hệ: chương 1 trước chương 2 vì chương 2 lắp vào chỗ chương 1 bỏ trống; chương 6 cuối cùng vì nó cần cả Λ từ chương 4 lẫn điều kiện khả nghịch từ chương 5.

Vì sao không rút ngắn được: đây là phần duy nhất trong cây mà tôi khẳng định như vậy, nên nó đáng biện minh. Mọi thứ ở bốn phần sau đều là các cách khác nhau để dùng sáu chương này. Một người bỏ qua A/03 sẽ đọc mọi Jacobian như bùa chú; một người bỏ qua A/05 sẽ chẩn đoán sai mọi lần hệ thống hỏng. Không có đường tắt, và cố đi tắt sẽ tốn nhiều thời gian hơn.

Cách đo tiến độ, và đây là điểm quan trọng: *tự dẫn lại được trên giấy*. Cụ thể: dẫn lại ràng buộc epipolar từ tính đồng phẳng; dẫn lại $\partial(R\mathbf{p})/\partial\phi$; dẫn lại phần bù Schur; dẫn lại bảy bậc tự do của SfM mono. Nếu chưa dẫn lại được thì chưa xong, bất kể đã đọc bao nhiêu lần.

Mini project nên làm: chương 1 (pipeline hai khung và ba cấu hình suy biến), chương 3 (kiểm mọi Jacobian bằng sai phân số — bài này tạo ra bộ unit test dùng suốt về sau), và chương 5 (phổ giá trị riêng trên bốn quỹ đạo).

Thời gian: đây là phần dài nhất và nên là phần dài nhất.

1.3 Giai đoạn 2 – Đọc hệ thống thật

Nội dung: đọc B/07 tới B/12 song song với việc đọc mã nguồn ba hệ thống, theo thứ tự ở E/23 mục 2: VINS-Fusion, rồi OpenVINS, rồi ORB-SLAM3.

Vì sao thứ tự hệ thống này: VINS-Fusion ngắn nhất và cấu trúc rõ nhất, nên nó là chỗ học cách đọc mã SLAM; OpenVINS cho thể giới bộ lọc và — quan trọng hơn — cho một bộ mô phỏng có chân trị, thứ cần cho C/18; ORB-SLAM3 đầy đủ nhất và phức tạp nhất, nên đọc nó cuối.

Bài tập bắt buộc cho mỗi hệ, và đây là phần biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm: *vẽ lại đồ thị nhân tử và liệt kê chính xác biến nào được fix*. Với mỗi hệ, trả lời: những biến nào trong vector trạng thái? Những loại nhân tử nào? Cái gì bị fix và vì sao? Nó xử lý gauge thế nào?

Bài tập ấy có giá trị vượt xa việc hiểu ba hệ thống: nó buộc bạn đặt mỗi hệ vào Bảng ở 00-map, và sau ba lần bạn sẽ thấy chúng là ba ô của cùng một bảng.

Cách đo tiến độ: bạn giải thích được vì sao mỗi hệ đưa ra các lựa chọn kiến trúc của nó, và bạn chỉ ra được ít nhất một chỗ mà mã khác với bài báo.

1.4 Giai đoạn 3 – Hợp nhất với hiệu chuẩn

Nội dung: C/14 cùng mini project của nó, cộng C/13 và C/15.

Vì sao giai đoạn này đứng riêng: vì nó là chỗ luận đề trung tâm của cây được chứng minh bằng tay. Mini project ở C/14 — viết một bộ giải rồi chạy bốn bài toán khác nhau chỉ bằng cách đổi danh sách biến — là bài tập làm sáng tỏ luận đề rõ nhất trong cả cây.

Quy trình đề nghị: chạy Kalibr trên dữ liệu của mình để có cảm giác; rồi tự viết một bài toán hiệu chuẩn bằng Ceres hoặc GTSAM; rồi *thêm dần biến vào* cho tới khi nó trở thành VIO. Mỗi lần thêm một biến, hỏi: biến này cần chuyển động gì để quan sát được? Đó là Bảng kích thích ở A/05 được kiểm bằng tay.

Cách đo tiến độ: bạn có một bộ mã chạy được cả bốn cấu hình, và bạn giải thích được phổ giá trị riêng của từng cấu hình.

Vì sao C/13 thuộc giai đoạn này: vì quản lý bản đồ và hiệu chuẩn chia nhau một câu hỏi — cái gì được coi là cố định và cái gì được phép đổi. Và vì chương 13 là chương bị coi nhẹ nhất, nên đặt nó cạnh một chương mà người học đang hào hứng là cách để nó không bị bỏ.

1.5 Giai đoạn 4 – Đối mặt thực tế

Nội dung: C/18 (phương pháp đánh giá) và D/19 (chế độ hồng), rồi tự dựng dữ liệu trong môi trường mục tiêu, đo thất bại, phân loại nguyên nhân.

Đây là giai đoạn mà tôi cho là quan trọng nhất và bị bỏ qua nhiều nhất, nên nó đáng một lập luận đầy đủ.

A. Vì sao nó bị bỏ qua

Ba lý do, và cả ba đều dễ hiểu.

Một — nó không có cảm giác tiến bộ. Đọc một bài báo mới cho cảm giác học được điều gì đó; thu ba giờ dữ liệu trong hành lang văn phòng thì không.

Hai — nó không có sản phẩm khoe được. Không có bảng số nào để đưa vào báo cáo, không có thuật toán mới nào.

Ba — nó đòi hỏi thừa nhận rằng mình chưa biết vấn đề là gì. Sau ba giai đoạn học, cảm giác tự nhiên là bắt tay vào cải tiến. Việc dừng lại để đo là một sự thừa nhận rằng ta chưa biết cần cải tiến cái gì.

B. Vì sao nó quan trọng nhất

Lập luận gọn: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ.*

Cụ thể hơn. Sau ba giai đoạn, bạn có một danh sách các thứ có thể cải tiến — front-end tốt hơn, back-end tốt hơn, bản đồ tốt hơn. Danh sách ấy đến từ việc đọc, nên nó phản ánh *những gì cộng đồng đang quan tâm*, không phản ánh *vấn đề của bạn*.

Trong thực tế, khi đo, người ta thường phát hiện rằng vấn đề thật là một thứ không có trong danh sách: một chỗ trong môi trường có gương; ánh sáng thay đổi vào một giờ nhất định; bánh xe trượt trên một loại sàn cụ thể; hiệu chuẩn trôi sau hai giờ chạy. Không vấn đề nào trong số đó được giải bằng một detector tốt hơn.

Theo C/18 mục 5D: *SOTA thật nằm ở đuôi phân bố*. Và đuôi phân bố của *bạn* chỉ đo được trên dữ liệu của bạn.

C. Làm gì cụ thể

Bước một — dựng bộ dữ liệu, theo ba phần ở D/21 mục 4A: chuỗi đại diện (dài, môi trường điển hình), chuỗi nghịch cảnh (theo mini project D/19), và — nếu có thiết bị chạy thật — chuỗi từ hiện trường.

Bước hai — đo theo bốn câu hỏi ở C/18 khối Cần nhớ: chính xác tới đâu, có thành thật không, hồng thế nào và bao nhiêu lần, ở đuôi thì sao.

Bước ba — phân loại nguyên nhân, không phân loại triệu chứng. Mười lần mất bám có thể có ba nguyên nhân, và việc gộp chúng theo triệu chứng che mất điều đó. Dùng các đại lượng ở D/19 khối Cần nhớ để phân cụm.

Bước bốn — chỉ bây giờ mới bắt đầu cải tiến, và bắt đầu từ cụm lớn nhất.

Cách đo tiến độ: bạn có một bảng xếp hạng nguyên nhân thất bại trong môi trường của bạn, với tần suất. Nếu bảng ấy trùng với danh sách bạn đoán trước khi đo thì hoặc bạn rất giỏi hoặc bạn chưa đo đủ khó.

1.6 Giai đoạn 5 — Nghiên cứu

Nội dung: D/22 để chọn ứng viên, rồi `research-rules.md` để làm đúng cách.

Quy trình, và thứ tự quan trọng.

Một — chọn một hướng, từ D/22 hoặc — tốt hơn — từ bảng xếp hạng nguyên nhân ở giai đoạn 4. Hướng đến từ vấn đề thật của bạn có ba lợi thế: bạn có dữ liệu, bạn có tiêu chí thành công, và bạn biết nó đáng giải.

Hai — chạy phép kiểm của hướng đó (D/22 khối *Cần nhớ*). Vài ngày, và nó có thể tiết kiệm một năm.

Ba — nếu phép kiểm xác nhận, nâng lên thành khảo sát đúng chuẩn: `research-rules.md` bước 0 tới bước 4. Đọc 20–30 công trình gần nhất của riêng hướng đó, có nhật ký tìm kiếm, có ma trận tổng hợp.

Bốn — tái hiện hai tới ba baseline trước khi đề xuất bất cứ gì. Theo C/18 mục 5B: nếu baseline của bạn không đạt con số công bố thì mọi so sánh về sau vô nghĩa. Và bước này thường dạy nhiều hơn cả bước năm.

Năm — chỉ bây giờ mới đề xuất.

Cách đo tiến độ: bạn có một `00-question.md` theo `research-rules.md` mục 7 — một câu hỏi đủ cụ thể để có đáp án đúng/sai, một giả thuyết kèm mức tin bằng số, và một tiêu chí falsify viết trước khi chạy.

CẦN NHỚ

Lộ trình rút gọn, nếu bạn chỉ có thời gian cho phần xương sống.

Bảy chương phải đọc kỹ và tự dẫn lại được: A/03 (Lie group), A/04 (MAP và BA), A/05 (observability), B/07 (front-end), B/09 (paradigm back-end), C/13 (quản lý bản đồ), C/14 (hiệu chuẩn).

Ba mini project có tỉ lệ giá trị trên thời gian cao nhất: kiểm Jacobian bằng sai phân số (A/03 — tạo ra bộ test dùng suốt); phổ giá trị riêng trên bốn quỹ đạo (A/05 — làm observability thành thứ nhìn thấy được); và đi từ hiệu chuẩn tới SLAM bằng cách thả biến (C/14 — chứng minh luận đề trung tâm).

Một hệ thống đọc trọn vẹn: VINS-Fusion, vì nó ngắn nhất mà vẫn đủ các thành phần.

Một việc không được bỏ: giai đoạn 4. Kể cả khi rút gọn mọi thứ khác, hãy tự thu dữ liệu trong môi trường của mình và đo thất bại. Đây là việc phân biệt một người hiểu lĩnh vực với một người hiểu *bài toán của mình* — và chỉ có cái thứ hai mới tạo ra thứ dùng được.

1.7 Một gợi ý về chỗ bắt đầu

Một nhận xét cuối, và nó là một nhận xét về phương pháp chứ không về nội dung.

Nếu bạn đang có một bài toán ứng dụng cụ thể — ví dụ docking bằng marker với yêu cầu độ chính xác milimet, chủ đề của `../marker/` — thì bài toán ấy thường là chỗ tốt hơn để bắt đầu so với việc đi tuần tự qua cây này.

Lý do: một bài toán ứng dụng hẹp chứa *bản thu nhỏ* của gần như toàn bộ lộ trình. Nó có hiệu chuẩn (C/14); nó có observability và lưỡng nghĩa (A/05); nó có ngân sách sai số (A/06); nó có yêu cầu sản xuất (D/21); và nó có một tiêu chí thành công rõ ràng mà không benchmark nào cho bạn.

Quan trọng hơn cả: nó cho bạn *lý do để cần* mỗi chương. Học A/05 vì tò mò thì khó nhớ; học nó vì tư thế của bạn đang lưỡng nghĩa và bạn cần biết vì sao thì nhớ rất lâu. Đây là nguyên tắc “học đúng lúc cần” ở mục 1, áp dụng ở quy mô lớn nhất.

Cách kết hợp đề nghị: dùng bài toán ứng dụng làm *trực*, và dùng cây này làm *tài liệu tra* — mỗi lần gặp một chỗ không hiểu, tìm chương tương ứng và đọc nó trọn vẹn. Sau vài tháng bạn sẽ đã đọc phần lớn cây, và mỗi chương đều gắn với một vấn đề thật bạn từng gặp.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba quan hệ phụ thuộc trong giai đoạn 1 và lý do logic của mỗi quan hệ.
2. Cách đo tiến độ của giai đoạn 1 là gì, và vì sao “đã đọc xong” không phải một cách đo?
3. Bài tập bắt buộc khi đọc mỗi hệ thống ở giai đoạn 2 là gì, và nó cho bạn thấy điều gì sau ba lần?
4. Nêu ba lý do vì sao giai đoạn 4 bị bỏ qua, và lập luận vì sao nó quan trọng nhất.
5. Vì sao một hướng nghiên cứu đến từ bảng xếp hạng nguyên nhân của bạn tốt hơn một hướng chọn từ danh sách?
6. Trong quy trình năm bước của giai đoạn 5, bước nào thường dạy nhiều nhất và vì sao?
7. Vì sao một bài toán ứng dụng cụ thể thường là chỗ tốt hơn để bắt đầu so với đi tuần tự qua cây?